

- ```
>>> "der Fluß".lower()
```
- ```
'der fluß'
```
- 
- ```
>>> "der Fluß".casefold()
```
- ```
'der fluss'
```
- In order to search a character sequence in a string, `find()` method comes in handy. It accepts a string as a parameter and searches the string for its presence. If found, returns the `start index` of the first occurrence else returns `-1`.
- ```
>>> s = "my name is david. david goliath"
```
- ```
>>> s.find("david") #returns start index of first match i.e first 'd'
```
- ```
11
```

We can anchor the search to the start or the end of the target string using `startswith()` or `endswith()` method respectively, which will return `True` if the substring is found, `False` otherwise. We can even limit the search to a `start` or an `end` index within the target string, by passing them as an optional parameter. All the aforementioned methods are case sensitive search operations.

- ```
>>> s = "I want to test this string"
```
- ```
>>> s.startswith("I")
```
- ```
True
```
- ```
>>> s.startswith("i") #Case mismatch, hence returns false
```
- ```
False
```
- ```
>>> s.startswith("Iwant") # A missing ' ' between I & want
```
- ```
False
```
- ```
>>> s.endswith("ing");
```
- ```
True
```

- String manipulation is another common operation that can be simplified using string methods. If we want to replace instances of a substring within a string we can use `replace(old, new)` method which takes the old and the new string for substitution.